Hedging - Volatility Mismatch

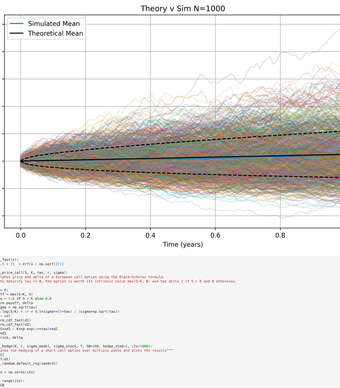
In [1]:
```
import numpy as np
from matplotlib import pyplot as plt
import scipy.stats as stats
from math import pi
```

1. Matching Volatility

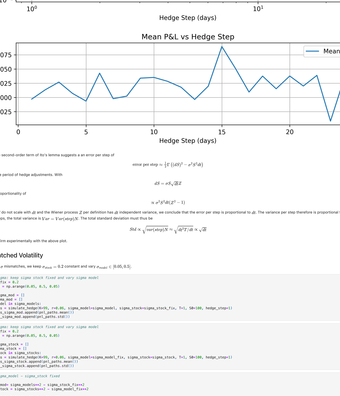
We firstly validate our GBM approach and then implement the delta hedge via Black-Scholes

In [2]:
```
def option_vega(S, r, sigma, dt, rng):
    """Calculates one Euler-Maruyama step of the option price"""
    Z = rng.standard_normal()
    return S*np.exp(r*dt)*sigma*np.sqrt(dt)

def validate_option_step():
    """Validates the option_step function by simulating multiple paths and plotting the results"""
    S = 100
    r = 0.05
    sigma = 0.2
    T = 1
    dt = 0.01
    N = int(T/dt)
    rng = np.random.default_rng(seed=42)
    ITS = 1000
    # Simulate multiple paths
    time_grid = np.linspace(0, T, N)
    for i in range(ITS):
        S = 100
        for t in range(N):
            S = S + option_vega(S, r, sigma, dt, rng)
            paths[i, t] = S

    # Theory
    time_grid = np.linspace(0, T, N)
    theoretical_mean = S0 * np.exp(r * time_grid)
    theoretical_std = S0 * np.exp(r * time_grid) * sigma * np.sqrt(time_grid)

    # Plot the results
    plt.figure(figsize=(8, 5), dpi=300)
    plt.plotting_linewidth(2, 0), paths, alpha=0.2)
    plt.plotting_linewidth(3, 0), theoretical_mean, label='Simulated Mean')
    plt.plotting_linewidth(3, 0), theoretical_mean, label='Theoretical Mean', color='black')
    plt.plotting_linewidth(3, 0), theoretical_mean - theoretical_std, color='black', ls='--')
    plt.plotting_linewidth(3, 0), theoretical_mean + theoretical_std, color='black', ls='--')
    plt.title('Theory v Sim N=1000')
    plt.legend()
    plt.xlabel('Time (years)')
    plt.ylabel('Option Price')
    plt.grid()
    plt.show()

    print("Theory: {theoretical_mean[-1]:.2f} ± {theoretical_std[-1]:.2f}")
    print("Sim: {paths[:,-1].mean():.2f} ± {paths[:,-1].std():.2f}")
    print("N = {ITS}")

validate_option_step()
```

```
Theory  100.50 ± 21.24
Sim     100.42 ± 21.25
N = 1000
```

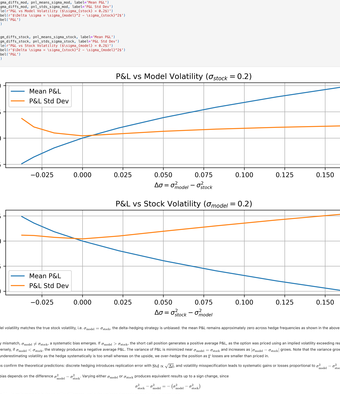


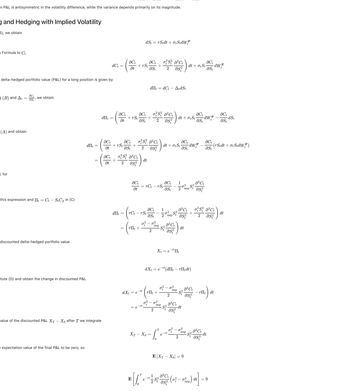
In [3]:
```
def norm_cdf(x):
    return 0.5 * (1 + erfc / np.sqrt(2)))

def bs_delta_price_call(S, K, tau, r, sigma):
    """Calculates price and delta of a European call option using the Black-Scholes formula.
    At time to maturity tau = 0, the option is worth its intrinsic value max(S-K, 0) and has delta 1 if S > K and 0 otherwise.
    """
    if tau == 0:
        return max(S-K, 0)
        delta = 1 if S > K else 0.0
    return price*T, delta
    sigma_sqrt_tau = sigma * np.sqrt(tau)
    d1 = (np.log(S/K) + (r + 0.5*sigma**2)*tau) / (sigma*np.sqrt(tau))
    d2 = d1 - sigma_sqrt_tau
    nd1 = norm_cdf(d1)
    nd2 = norm_cdf(d2)
    price = S * nd1 - K*np.exp(-r*tau)*nd2
    delta = nd1
    return price, delta

def simulate_hedge(N, r, sigma_model, sigma_stock, T, S0=100, hedge_step=1, ITS=1000):
    """Simulates the hedging of a short call option over multiple paths and plots the results"""
    dt = T/N
    N = int(N)
    rng = np.random.default_rng(seed=42)

    pnl_paths = np.zeros(ITS)

    for i in range(ITS):
        S = S0

        # t = 0
        price, delta = bs_delta_price_call(S, K, T, r, sigma_model)
        cash = option_price - delta * S # initial cash from selling the option and buying the delta hedge

        for t in range(N):
            # one step stock
            S = S + option_vega(S, r, sigma_stock, dt, rng)
            # accrue interest on cash
            cash *= np.exp(r*dt)

            # rebalance hedge at hedge_step intervals
            if (t+1) % hedge_step == 0 and t != N-1:
                tau = N*dt - (t+1)*dt, # DO
                price, new_delta = bs_delta_price_call(S, K, tau, r, sigma_model)
                # Rebalance the hedge
                cash += (new_delta - delta) * S # Adjust cash for the change in delta
                delta = new_delta

        # Payoff of the short option
        payoff = max(S-K, 0)
        # Final P&L: receive payoff and closing the hedge position
        pnl_paths[i] = cash + delta * S - payoff

    return pnl_paths

def plot_pnl_distribution(pnl_paths, ITS, hedge_step):
    """Plots the distribution of short call P&L from the hedging simulation"""
    plt.figure(figsize=(8, 5), dpi=300)
    plt.hist(pnl_paths, bins=50, density=True, alpha=0.5, color='r')
    plt.title('P&L Distribution of Short Call Hedge (N=1000, Hedge Step={hedge_step} days)')
    plt.xlabel('P&L')
    plt.ylabel('Density')
    plt.grid()
    plt.show()

N1 = simulate_hedge(N=N1, r=0.05, sigma_model=0.2, sigma_stock=0.2, T=1, S0=100, hedge_step=1)
plot_pnl_distribution(pnl=N1, ITS=1000, hedge_step=1)
```



P&L distribution for initial given values of the short hedge. Assumed volatilities are equal for model and stock. Hedge balance every single day.

In [4]:
```
hedge_steps = [ *for* i *in* range(1, 25)]
pnl_stds = []
pnl_slope = []

for hedge_step in hedge_steps:
    pnl_paths = simulate_hedge(N=N1, r=0.05, sigma_model=0.2, sigma_stock=0.2, T=1, S0=100, hedge_step=hedge_step)
    pnl_means.append(pnl_paths.mean())
    pnl_stds.append(pnl_paths.std())

fig = plt.figure(figsize=(8, 6), dpi=300)
gs = fig.add_gridspec(2, 1, hspace=0.5)
axs = fig.add_subplots(gs)
axs[0 + 0].subplot(gs[0])

coef = np.polyfit(np.log(hedge_steps), np.log(pnl_stds), 1)
slope = coef[0]
axs1.loglog(hedge_steps, pnl_stds, label='P&L Std Dev')
axs1.loglog(hedge_steps, np.exp(coef[1]) * hedge_steps**slope, label=f'Fit: $\sigma^{{{slope:.2f}}}$', ls='--')
axs1.set_title('Std Dev of P&L vs Hedge Step')
axs1.set_xlabel('Hedge Step (days)')
axs1.set_ylabel('P&L')
axs1.legend()
axs1.grid()

axs2.plot(hedge_steps, pnl_means, label='Mean P&L')
axs2.set_title('Mean P&L vs Hedge Step')
axs2.set_xlabel('Hedge Step (days)')
axs2.set_ylabel('P&L')
axs2.legend()
axs2.grid()
```





Recall that the second-order term of Ito's lemma suggests a an error per step of

$$\text{error per step} = \left[\sigma^2\left(\Delta S\right)^2 - \sigma^2 S^2 dt\right]$$

where ΔS is the period of hedge adjustments. With

$$dS = \sigma S_t \sqrt{\Delta t}$$

we obtain a proportionality of

$$\propto \sigma^2 S^2 dt \left(Z^2 - 1\right)$$

and since $\mathbb{E}_{\Delta}$ $\mathbb{E}$ not scale with Δt and the Wiener process Z per definition has Δt independent variance, we conclude that the error per step is proportional to Δt. The variance per step therefore is proportional to Δt^2. For $N = T/\Delta t$ steps, the total variance is $\propto T\sigma^4\mathbb{E}\left[\left(Z^2-1\right)^2\right]\Delta t$. The total standard deviation must thus be

$$\text{Std} \propto \sqrt{\text{Var}[\text{error}]N} \propto \sqrt{\Delta t^2 T} \propto \sqrt{\Delta t}$$

which we confirm experimentally with the above plot.

2. Mismatched Volatility

To investigate μ mismatches, we keep $\sigma_{\text{model}} = 0.2$ constant and vary $\sigma_{\text{stock}} \in [0.05, 0.5]$:

In [5]:
```
# varying sigma: keep sigma_stock fixed and vary sigma_model
sigma_fix = 0.2
sigma_model = np.arange(0.05, 0.5, 0.05)

pnl_means_sigma_mod = []
pnl_stds_sigma_mod = []
for sigma_model in sigma_model:
    pnl_paths = simulate_hedge(N=N1, r=0.05, sigma_model=sigma_model, sigma_stock=sigma_stock_fix, T=1, S0=100, hedge_step=1)
    pnl_means_sigma_mod.append(pnl_paths.mean())
    pnl_stds_sigma_mod.append(pnl_paths.std())

# varying sigma: keep sigma_stock fixed and vary sigma_model
sigma_model_fix = 0.2
sigma_stocks = np.arange(0.05, 0.5, 0.05)

pnl_means_sigma_stock = []
pnl_stds_sigma_stock = []
for sigma_stock in sigma_stocks:
    pnl_paths = simulate_hedge(N=N1, r=0.05, sigma_model=sigma_model_fix, sigma_stock=sigma_stock, T=1, S0=100, hedge_step=1)
    pnl_means_sigma_stock.append(pnl_paths.mean())
    pnl_stds_sigma_stock.append(pnl_paths.std())
```

In [6]:
```
# plot sigma_model - sigma_stock fixed

sigma_diffs_model = sigma_model - sigma_stock_fixed
sigma_diffs_stock = sigma_stocks - sigma_model_fixed

fig = plt.figure(figsize=(8, 6), dpi=300)
axs1 = fig.add_gridspec(2, 1, hspace=0.5)
axs1 = plt.subplots(gs[0])
axs2 = plt.subplots(gs[1])

axs1.plot(sigma_diffs_mod, pnl_means_sigma_mod, label='Mean P&L')
axs1.plot(sigma_diffs_mod, pnl_means_sigma_mod - pnl_stds_sigma_mod, ls='--')
axs1.set_title('P&L vs Model Volatility ($\sigma_{stock}$ = 0.2)')
axs1.set_xlabel('$\Delta\sigma$ Sigma = $\sigma_{model}$ - $\sigma_{stock}$')
axs1.legend()
axs1.grid()

axs2.plot(sigma_diffs_stock, pnl_means_sigma_stock, label='Mean P&L')
axs2.plot(sigma_diffs_stock, pnl_means_sigma_stock - pnl_stds_sigma_stock, ls='--')
axs2.set_title('P&L vs Stock Volatility ($\sigma_{model}$ = 0.2)')
axs2.set_xlabel('$\Delta\sigma$ Sigma = $\sigma_{model}$ - $\sigma_{stock}$')
axs2.set_ylabel('P&L')
axs2.legend()
axs2.grid()
```

When the model volatility matches the true stock volatility, i.e $\sigma_{\text{model}} = \sigma_{\text{stock}}$, the delta-hedging strategy is unbiased: the mean P&L remains approximately zero across hedge frequencies as shown in the above experiments.

Under volatility mismatch, $\sigma_{\text{model}} \neq \sigma_{\text{stock}}$, a systematic bias emerges. If $\sigma_{\text{model}} > \sigma_{\text{stock}}$, the short call position generates a positive average P&L, as the option was priced using an implied volatility exceeding realized volatility. Conversely, if $\sigma_{\text{model}} < \sigma_{\text{stock}}$, the strategy produces a negative average P&L. The variance of P&L is minimized near $\sigma_{\text{model}} = \sigma_{\text{stock}}$ and increases as $|\sigma_{\text{model}} - \sigma_{\text{stock}}|$ grows. Note that the variance grows more quickly when underestimating volatility as the hedge systematically is too small whereas on the upside, we over-hedge the position as Δ losses are smaller than priced in.

The simulations confirm the theoretical predictions: discrete hedging introduces replication error with $\text{Std} \propto \sqrt{\Delta t}$, and volatility misspecification leads to systematic gains or losses proportional to $\sigma_{\text{model}}^2 - \sigma_{\text{stock}}^2$.

Note that the bias depends on the difference $\sigma_{\text{model}}^2 - \sigma_{\text{stock}}^2$ (varying within σ_{model} or σ_{stock} produces equivalent results up to a sign change, since

$$\sigma_{\text{stock}}^2 - \sigma_{\text{model}}^2 = -\left(\sigma_{\text{model}}^2 - \sigma_{\text{stock}}^2\right)$$

Thus, the mean P&L is antisymmetric in the volatility difference, while the variance depends primarily on its magnitude.

3. Pricing and Hedging with Implied Volatility

Via equation (B), we obtain

$$d\Pi_t = r\Pi_t dt + \sigma_t S_t dW_t^{\mathbb{Q}} \tag{A}$$

and apply Itô's Formula to C_t

$$dC_t = \left(\frac{\partial C_t}{\partial t} + rS_t\frac{\partial C_t}{\partial S_t} + \frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}\right)dt + \sigma_t S_t \frac{\partial C_t}{\partial S_t} dW_t^{\mathbb{Q}} \tag{B}$$

The change in delta-hedged portfolio value ($P\&L$) for a long position is given by:

$$d\Pi_t = dC_t - \Delta_t dS_t$$

by substituting (B) and $\Delta_t = \frac{\partial C_t}{\partial S_t}$, we obtain

$$d\Pi_t = \left(\frac{\partial C_t}{\partial t} + rS_t\frac{\partial C_t}{\partial S_t} + \frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}\right)dt + \sigma_t S_t \frac{\partial C_t}{\partial S_t} dW_t^{\mathbb{Q}} - \frac{\partial C_t}{\partial S_t} dS_t$$

We substitute (A) and obtain

$$d\Pi_t = \left(\frac{\partial C_t}{\partial t} + rS_t\frac{\partial C_t}{\partial S_t} + \frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}\right)dt + \sigma_t S_t \frac{\partial C_t}{\partial S_t} dW_t^{\mathbb{Q}} - \frac{\partial C_t}{\partial S_t}(rS_t dt + \sigma_t S_t dW_t^{\mathbb{Q}})$$

$$= \left(\frac{\partial C_t}{\partial t} + \frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}\right)dt \tag{C}$$

We arrange (A) for

$$\frac{\partial C_t}{\partial t} = r\Pi_t - rS_t\frac{\partial C_t}{\partial S_t} - \frac{1}{2}\sigma_t^2 S_t^2 \frac{\partial^2 C_t}{\partial S_t^2}$$

and substitue this expression and $\Pi_t = C_t - S_t C_t = 0$ in (C)

$$d\Pi_t = \left(rC_t - rS_t\frac{\partial C_t}{\partial S_t} - \frac{1}{2}\sigma_{\text{mkt}}^2 S_t^2 \frac{\partial^2 C_t}{\partial S_t^2} + \frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}\right)dt$$

$$= \left(r\Pi_t + \frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}\right)dt \tag{D}$$

Let X_t be the discounted delta-hedged portfolio value

and thus

$$dX_t = e^{-rt} d\Pi_t$$

We now substitute (D) and obtain the change in discounted P&L:

$$dX_t = e^{-rt}\left(r\Pi_t + \frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}\right)dt = e^{-rt}\frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}dt \tag{E}$$

To obtain the value of the discounted P&L $X_T - X_0$ after T we integrate

$$X_T - X_0 = \int_0^T e^{-rt}\frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}dt$$

We require the expectation value of the final P&L to be zero, so

and thus

$$\mathbb{E}\left[\int_0^T e^{-rt}\frac{\sigma_t^2 S_t^2}{2}\frac{\partial^2 C_t}{\partial S_t^2}\left(\sigma_t^2 - \sigma_{\text{mkt}}^2\right)dt\right] = 0$$

Note that this is the expectation value for a long position and the signs of the γ terms differ (from equation (D) given in the problem set. (D) corresponds to the P&L value of a short position

$$\Pi_t = -C_t + \Delta_t S_t = -\Pi_t$$

while we assume a long position. Multiplying our result by (-1), we arrive at equation (E):